



Sistemas Embebidos

Proyecto Final

Integrantes del grupo:

Sebastián Berardi
Gervasio Hernández
Joaquín Cuitiño

Repo: <https://github.com/epicgera/Lab1IoT>

Release: https://github.com/epicgera/Lab1IoT/releases/tag/Lab_FIN

Docentes: Nicolás Calarco y Pablo Alonso

03/05/2025

Objetivo	3
Introducción	3
Desarrollo	4
PLAYER.....	5
PLAYLIST.....	7
LOGGER	9
MQTT.....	11
DELAY	13
LED	13
LED_STRIP	13
TOUCH.....	14
WIFI	14
HTTP	16
NTP	18
Bibliografía.....	19

Objetivo

El objetivo del proyecto es generar un sistema embebido que sea capaz de reproducir canciones y controlar remotamente las acciones de una lista de reproducción local a través del protocolo MQTT.

Introducción

El proyecto consiste en guardar una lista de reproducción (cierta cantidad de “temas”) en la memoria flash del dispositivo, que posea las siguientes instrucciones de control posibles: ir al tema siguiente, al anterior, subir y bajar el volumen, play/pausa y detener. La salida de sonido debe ser a través de la placa de extensión de audio: ESP-LyraT-8311A v1.2, tanto por el parlante que viene en el kit como a través del jack de audio. Las instrucciones del control de reproducción deberán poder recibirse tanto por protocolo mqtt como a través de una página web de configuración general, en la que también deberán poder configurarse los parámetros de comunicación mqtt y la configuración de conexión de la STA, tener los controles de reproducción y la posibilidad de agregar o quitar canciones. Nota: Deben utilizarse semáforos cuando los recursos sean compartidos, sin excepción.

Reproducción

La reproducción debe llevarse a cabo a través de una task que vaya ejecutando los comandos desde una queue que reciba instrucciones de todas las alternativas posibles (MQTT, Webserver, Touchpad, etc).

Conectividad

El dispositivo deberá funcionar como AP (access point) + STA (station) y ser host de una página web sencilla desde la cual se podrá configurar la red a la cuál conectarse en modo STA. Es suficiente que se configure el SSID y contraseña utilizando DHCP. No es necesario soportar IP estática. También desde esta página debe poder configurarse el broker mqtt y permitir accionar los controles de reproducción.

Logger

Deberá almacenar en un buffer circular de 20 espacios, en memoria no volátil, los eventos de reproducción (play/pausa, siguiente, anterior y detener) que vayan sucediendo.

MQTT

En la misma página web en la que se configura la conexión WiFi, se deberá poder configurar también la dirección del broker MQTT al que se conectará el dispositivo (solamente URL o IP, y puerto). Una vez que el dispositivo logre establecer la conexión MQTT con el broker, deberá enviar todos los eventos registrados en el logger en orden cronológico (las más viejas primero).

Las instrucciones almacenadas se deben enviar con QoS 1, y solo se considerarán enviadas una vez que se reciba el acknowledge correspondiente. El paquete debe armarse en formato JSON. Se sugiere utilizar la biblioteca cJSON.

Broker MQTT

Para testear la conexión MQTT del dispositivo, se puede utilizar cualquier broker MQTT. Se sugiere utilizar un broker local (por ejemplo, mosquitto) para facilitar el debugging y la visualización de información, pero también puede ser un broker online público (por ejemplo <https://www.hivemq.com/public-mqtt-broker/>).

NTP

La hora del sistema se deberá mantener sincronizada contra un servidor NTP a través de la conexión WiFi. Esto es necesario para que el logger almacene los datos con la hora correcta. Se recomienda sincronizar usando pool.ntp.org.

Configuración general

Las configuraciones de la aplicación y las canciones deberán estar grabadas en la memoria no volátil, de modo de que, si se corta la energía de la placa, al restablecerse pueda volver a funcionar sin necesidad de flashear nuevamente.

Desarrollo



Reproducción

Para la parte de conectividad se crearon dos componentes, uno llamado player y otro llamado playlist. Cabe resaltar que además se adiciono un componente ya creado por ESP-IDF para el codec ES8311, al cual lo único que se le cambio fue la configuración de algunos puertos para que quedara compatible con la placa de versión 1.3, ya que originalmente este estaba diseñado para la versión 1.2.

PLAYER

Este componente implementa un reproductor de audio, que gestiona la reproducción de canciones mediante una cola de comandos y controla el hardware de audio (codec ES8311 e I2S). Inicializa el hardware necesario, administra comandos como reproducir, pausar, detener, avanzar o retroceder canción, y controla el volumen. La reproducción ocurre en una tarea dedicada que lee datos de audio en chunks y los envía vía I2S al DAC. Además, hay una tarea “heartbeat” que indica con un LED el estado actual (reproduciendo o no). El acceso a variables compartidas se protege con un mutex, y los comandos se envían a través de una cola para sincronización y control.

Estructura del código:

audio_task (función principal de reproducción):

Repetir por siempre:

Si llega un comando en la cola (con timeout 10ms):

Según el comando recibido:

CMD_PLAY:

Si no está reproduciendo:

Si hay stream abierto:

Continuar reproducción desde posición actual

Sino:

Abrir stream para la canción actual y empezar reproducción

CMD_PAUSE:

Marcar estado como no reproduciendo

CMD_STOP:

Si hay stream abierto:

Cerrar stream

Marcar estado como no reproduciendo

CMD_NEXT:

Si hay stream abierto:

Cerrar stream

Avanzar a la siguiente canción en playlist

Si se pudo abrir stream:

Empezar reproducción

Sino:

Marcar estado como no reproduciendo

CMD_PREV:

Si hay stream abierto:

Cerrar stream

Retroceder a la canción anterior en playlist

Si se pudo abrir stream:

Empezar reproducción

Sino:

Marcar estado como no reproduciendo

CMD_VOL_UP:

Incrementar volumen (hasta 100) y aplicar al codec

CMD_VOL_DOWN:

Decrementar volumen (hasta 0) y aplicar al codec

```
Si está reproduciendo y el stream está abierto:
  Leer chunk de audio de la playlist
  Si lectura correcta y hay datos:
    Escribir chunk al canal I2S para reproducción
    Si error en escritura:
      Parar reproducción
  Si lectura correcta y sin datos (fin de canción):
    Cerrar stream
    Enviar comando CMD_NEXT para avanzar a siguiente canción
  Si error en lectura:
    Parar reproducción y cerrar stream si está abierto
Sino si no está reproduciendo:
  Esperar 100 ms para no consumir CPU innecesariamente
```

heartbeat_task (función para indicador LED):

```
Repetir por siempre:
  Si está reproduciendo:
    Encender LED verde 500ms
    Apagar LED 500ms
  Sino:
    Encender LED azul tenue 500ms
    Apagar LED 500ms
```

Funciones auxiliares:

```
player_init:
  Inicializar hardware, playlist, mutex, cola y tareas
  Retornar cola de comandos para uso externo
player_send_cmd:
  Enviar comando a la cola
player_set_volume:
  Actualizar variable volumen y configurar codec
player_get_volume:
  Retornar volumen actual
player_is_playing:
  Retornar estado de reproducción (bool)
```

init_player (función de arranque):

```
Inicializar driver I2S (i2s_driver_init):
  Configurar canales I2S TX y RX
  Configurar reloj, pines y modo I2S
  Habilitar canales I2S
Inicializar codec ES8311 (es8311_codec_init):
  Configurar I2C para comunicación con codec
  Crear handle del codec ES8311
  Configurar reloj y frecuencia de muestreo
  Configurar volumen inicial y micrófono deshabilitado
```

- Configurar GPIO para controlar amplificador de potencia (PA)
- Inicializar playlist
- Crear mutex para proteger acceso a variables críticas
- Crear cola de comandos para recibir instrucciones del usuario
- Crear tarea audio_task para manejar reproducción de audio
- Crear tarea heartbeat_task para controlar el LED indicador

PLAYLIST

Este componente gestiona una lista de reproducción de canciones almacenadas en el sistema de archivos SPIFFS del ESP32. Permite cargar la lista desde un archivo de texto, navegar entre canciones (siguiente, anterior), agregar o eliminar canciones, y abrir archivos de audio para su lectura en streaming. Además, maneja el estado del stream (archivo abierto, posición, tamaño) y permite leer fragmentos de audio secuencialmente. Está diseñado para integrarse con el reproductor principal, que consume datos del stream para reproducirlos.

Estructura del código:

```
playlist_init():
    Montar sistema de archivos SPIFFS
    Intentar abrir archivo playlist.txt
    Si no existe:
        Inicializar playlist vacía
    Sino:
        Leer línea por línea las rutas de las canciones
        Guardar cada línea como una canción en la lista
    Inicializar índice de canción actual en 0
    Retornar OK

playlist_get_count:
    Retornar cantidad actual de canciones cargadas

playlist_get_current_song:
    Si hay canciones:
        Retornar nombre de archivo de la canción actual
    Sino:
        Retornar NULL

playlist_get_current_index:
    Retornar índice actual

playlist_set_index(index):
    Si el índice es válido:
        Cerrar stream actual
        Establecer el nuevo índice como canción actual
    Sino:
        Retornar error

playlist_next:
    Si no hay canciones:
        Retornar error
```

- Cerrar stream actual
- Avanzar al siguiente índice circularmente
- Retornar OK

playlist_prev:

- Si no hay canciones:
 - Retornar error
- Cerrar stream actual
- Retroceder índice (con wraparound si es necesario)
- Retornar OK

playlist_save:

- Abrir archivo playlist.txt para escritura
- Si error:
 - Retornar FAIL
- Escribir todas las canciones línea por línea
- Cerrar archivo
- Retornar OK

playlist_add_song(nombre_archivo):

- Si hay espacio en la lista:
 - Agregar canción al final
 - Guardar lista con playlist_save
- Sino:
 - Retornar error por falta de espacio

playlist_remove_song(index):

- Si el índice es válido:
 - Eliminar canción desplazando las siguientes
 - Ajustar el índice actual si es necesario
 - Guardar lista con playlist_save
- Sino:
 - Retornar error

playlist_open_current_stream:

- Si el índice actual es inválido:
 - Retornar error
- Si hay un stream abierto:
 - Cerrar stream
- Armar ruta completa del archivo de la canción actual
- Intentar abrir archivo en modo binario
- Si no se pudo abrir:
 - Retornar FAIL
- Medir tamaño total del archivo
- Establecer posición en 0
- Si archivo vacío:
 - Cerrar archivo y retornar error
- Loggear éxito y retornar OK

playlist_read_chunk(buffer, tamaño, *leídos):


```
Si no hay stream abierto o buffer inválido:
    Retornar error
Leer chunk del archivo al buffer
Actualizar posición
Si fin de archivo:
    Retornar OK
Si error:
    Retornar FAIL
Retornar OK
```

```
playlist_close_stream:
    Si hay stream abierto:
        Cerrar archivo
        Reiniciar posición y tamaño
        Loggear cierre
    Retornar OK
```

```
playlist_seek_stream(offset):
    Si no hay stream:
        Retornar error
    Intentar mover puntero del archivo
    Si falla:
        Retornar FAIL
    Actualizar posición
    Retornar OK
```

```
playlist_get_stream_position:
    Retornar posición actual del stream
```

```
playlist_get_stream_size:
    Retornar tamaño total del stream
```

LOGGER

Este componente implementa un sistema de registro (logger) que almacena una cantidad limitada de eventos (20) en un buffer circular en memoria y los persiste en la memoria no volátil (NVS) del dispositivo. Este logger guarda un máximo de 20 eventos, manteniendo un índice “head” para la posición donde se escribirá el próximo evento y un contador de eventos actuales. Permite agregar eventos, recuperar el listado actual, imprimir el buffer y preservar estos datos entre reinicios mediante el almacenamiento en NVS. Además, incluye una inicialización que carga el estado previo del logger desde NVS o, si no existe, inicializa el buffer limpio y guarda esa información en NVS para futuras cargas.

Estructura del código:

```
logger_print_buffer():
    Imprimir head y count
    Calcular índice inicial de lectura (igual que en get_events)
    Para cada evento guardado en buffer:
        Imprimir índice y cadena representativa del evento
```

logger_save_head_and_count():

- Abrir NVS en modo lectura-escritura
- Guardar variable head con su clave
- Guardar variable count con su clave
- Confirmar cambios y cerrar NVS

logger_save_event(idx):

- Abrir NVS en modo lectura-escritura
- Crear clave de evento según índice (ej. "event0", "event1", ...)
- Guardar evento en NVS con la clave correspondiente
- Confirmar cambios y cerrar NVS

logger_load_from_nvs():

- Abrir NVS en modo solo lectura
- Leer variables head y count desde NVS
- Para cada índice de buffer:
 - Leer evento de NVS con clave correspondiente
 - Si falla, inicializar evento con valor 0
- Cerrar NVS

logger_add_event(evento):

- Guardar el evento en el buffer en la posición head
- Guardar ese evento individualmente en NVS usando su índice
- Incrementar head circularmente ($\text{head} = (\text{head} + 1) \% \text{tamaño buffer}$)
- Si $\text{count} < \text{tamaño buffer}$:
 - Incrementar count
- Guardar head y count en NVS
- Imprimir el contenido completo del buffer en consola

logger_get_events(eventos[], *out_count):

- Validar punteros de entrada
- Calcular índice inicial de lectura en buffer: $(\text{head} + \text{tamaño buffer} - \text{count}) \% \text{tamaño buffer}$
- Copiar eventos desde buffer circularmente en el arreglo eventos[]
- Guardar en out_count la cantidad actual de eventos almacenados

init_logger:

- Inicializar NVS flash
- Si la inicialización falla por falta de espacio o versión nueva:
 - Borrar NVS
 - Reintentar inicialización
- Intentar cargar buffer y variables head, count desde NVS:
 - Si falla:
 - Inicializar head y count en 0
 - Limpiar buffer de eventos
 - Guardar head y count en NVS
 - Guardar cada posición del buffer vacío en NVS
- Fin inicialización

MQTT

Para esta parte creamos un componente llamado `communicate_mqtt`. Este gestiona la comunicación MQTT de la placa. Se encarga de conectar a un broker MQTT, manejar eventos MQTT (conexión, recepción de mensajes, publicación confirmada), y administrar una cola de comandos recibidos por MQTT para controlar un reproductor multimedia.

Los mensajes que se publican están codificados en base64 y en formato JSON. Se mantiene un buffer local de eventos para reenviarlos automáticamente tras una reconexión al broker. También se soporta la configuración persistente (broker, puerto, topics) guardándola y cargándola desde la memoria flash (NVS). Además, hay una tarea que consume comandos desde una cola y ejecuta acciones relacionadas con la reproducción (play, pause, stop, next, prev, volumen), enviando estados de ejecución vía MQTT.

Estructura del código:

Definir variables globales para:

- configuración MQTT (broker, puerto, topics)
- buffer de eventos y cola de comandos MQTT
- estado de publicación y manejo JSON base64

`publish_json_base64(topic, json):`

- Codificar JSON a string sin formato
- Codificar string en base64
- Publicar mensaje base64 en topic MQTT
- Devolver id del mensaje publicado

`log_event_to_string(evento):`

- Convertir enum evento a string legible

`publicar_siguiete_evento():`

- Si quedan eventos JSON en buffer:
 - Publicar el siguiente evento base64
 - Marcar esperando confirmación
- Sino:
 - Liberar memoria del buffer JSON
 - Reiniciar índices y estado

`mqtt_event_handler(evento):`

- Según tipo de evento MQTT:
 - Si conectado:
 - Guardar cliente MQTT global
 - Suscribir al topic eventos
 - Publicar evento "Conexión establecida"
 - Si hay buffer de eventos pendientes:
 - Preparar buffer JSON con eventos
 - Publicar el primer evento
- Si datos recibidos:
 - Extraer topic y mensaje
 - Si topic es de eventos:

Si mensaje es "Enviar" y hay buffer:

Preparar buffer JSON con eventos

Publicar el primer evento

Si mensaje es comando válido (play, pause, stop, next, prev, volup, voldown):

Enviar comando a la cola de eventos

Sino:

Loguear comando inválido

Si mensaje publicado:

Si esperando confirmación:

Incrementar índice

Publicar siguiente evento

connect_mqtt(uri, puerto, topic_evento):

Construir mensaje LWT codificado en base64

Configurar cliente MQTT con broker, puerto y LWT

Iniciar cliente MQTT y registrar manejador de eventos

almacenar_eventos(cola, topic):

Guardar cola global y topic de eventos

enviar_eventos_buffe(buffer, tamaño, topic_buffer):

Copiar buffer local de eventos para reenviar

Guardar topic buffer para publicar

enviar_estado_mqtt(mensaje):

Si cliente MQTT conectado y topic válido:

Crear JSON con estado y timestamp

Publicar JSON en topic eventos

eventos_task:

Mientras siempre:

Esperar comando de la cola eventos_queue

Según comando recibido:

Ejecutar función player_send_cmd correspondiente (play, pause, stop, next, prev, volup, voldown)

Enviar estado MQTT confirmando ejecución del comando

Si comando desconocido:

Loguear y enviar estado MQTT con error

init_mqtt():

Intentar inicializar MQTT desde configuración flash

Si no existe configuración:

Crear cola de eventos

Almacenar cola y topic por defecto

Obtener eventos desde logger local y guardarlos en buffer

Conectar MQTT con broker, puerto y topic por defecto

Crear tarea eventos_task

Guardar configuración MQTT por defecto en flash

`mqtt_save_config(config):`

- Abrir NVS para escritura

- Guardar broker, puerto, topic_evento, topic_buffer en NVS

- Confirmar cambios y cerrar NVS

`mqtt_load_config(config):`

- Abrir NVS para lectura

- Cargar broker, puerto, topic_evento, topic_buffer desde NVS

- Cerrar NVS

`mqtt_clear_config():`

- Abrir NVS para escritura

- Borrar todos los datos del namespace MQTT

- Confirmar cambios y cerrar NVS

`mqtt_init_from_flash():`

- Intentar cargar configuración MQTT desde flash

- Si éxito:

 - Crear cola de eventos

 - Almacenar cola y topic cargado

 - Obtener eventos logger y guardar en buffer

 - Conectar MQTT con configuración cargada

 - Crear tarea eventos_task

- Retornar resultado

DELAY

Implementa dos funciones de retardo (`delay_ms` y `delay_s`) que permiten pausar la ejecución del programa por una cantidad específica de milisegundos o segundos. Estas funciones se basan en la función `esp_rom_delay_us`, que realiza un retardo en microsegundos. Así, `delay_ms` multiplica el valor en milisegundos por 1000, y `delay_s` lo multiplica por 1.000.000 para convertirlo a microsegundos.

LED

Gestiona el encendido de un LED RGB mediante una función principal llamada `set_led()` que recibe un color (rojo, verde, azul) y lo aplica al LED. Si la tira no está inicializada, la función primero la inicializa llamando a `led_rgb_init()` y luego configura el color del primer LED de la tira. Finalmente, refresca la tira para aplicar el nuevo color.

LED_STRIP

Define cómo inicializar, controlar, refrescar, borrar y liberar una tira de LEDs (aunque en este caso se usa solo uno). Se adapta la señal RMT a los valores de temporización del protocolo WS2812, convirtiendo los colores RGB a pulsos digitales compatibles. También proporciona funciones para establecer el color de un píxel, actualizar el estado del LED, apagarlo y liberar memoria. Finalmente, una función de inicialización

completa configura el canal RMT, crea la instancia del LED y apaga el LED inicialmente. (Este componente fue extraído directamente de los ejemplos de ESP-IDF)

TOUCH

Gestiona la interfaz táctil del dispositivo mediante botones capacitivos que utilizan los touch pads del Kaluga. El módulo configura y monitorea los botones para controlar las funciones principales del reproductor.

La estructura quedo de la siguiente manera:

`touch_init()`: Inicializar sistema de touch del ESP32

`touch_update()`: Actualizar estado de todos los botones y guardarlo en una variable

`touch_pressed(numero_botón)`: Verificar si botón específico está presionado

`touchpad_task(argumentos)`: Tarea principal de monitoreo táctil (utiliza `touch_update`)

`init_touch()`: Función de inicialización principal del módulo

WIFI

En el caso del este componente, se debió modificar considerablemente para que se pudiera editar el SSID y la contraseña desde la web. Quedando un componente el cual se encarga de manejar toda la lógica de inicialización y configuración de WiFi, incluyendo el guardado y carga de configuración desde la memoria flash (NVS). El dispositivo puede operar en dos modos: AP (Access Point) o STA (Station/cliente).

Cuando arranca, intenta cargar la configuración WiFi guardada. Si existe, se inicializa en el modo correspondiente (STA o AP). Si no hay configuración guardada o hay un error, arranca con una configuración AP por defecto (SSID: CALIOPE 2.0, Contraseña: 1234567890). Además, se maneja la reconexión automática en modo STA y si falla varias veces, el dispositivo cambia automáticamente a modo AP.

Estructura del código:

`restart_task`:

Mostrar mensaje de reinicio

Esperar 3 segundos

Llamar `esp_restart`

Fin tarea

`wifi_event_handler(evento)`:

Si el evento es `WIFI_EVENT`:

Si es `STA_START`:

Reiniciar contador de reintentos

Llamar `esp_wifi_connect`

Si es `STA_CONNECTED`:

Reiniciar contador de reintentos

Si es `STA_DISCONNECTED`:

Si contador de reintentos < `MAX_RETRY_ATTEMPTS`:

- Incrementar contador
- Llamar esp_wifi_connect
- Sino:
 - Guardar configuración AP por defecto en flash
 - Crear tarea restart_task para reiniciar en 3 segundos
- Si es AP_STACONNECTED:
 - Mostrar MAC y AID del cliente conectado
- Si es AP_STADISCONNECTED:
 - Mostrar MAC y AID del cliente desconectado
- Si el evento es IP_EVENT_STA_GOT_IP:
 - Mostrar IP obtenida

init_ap(ssid, password):

- Inicializar pila de red y eventos
- Crear interfaz WiFi en modo AP
- Inicializar WiFi
- Registrar manejador de eventos WiFi
- Configurar estructura de AP con SSID, password, canal y autenticación
- Si el password está vacío, usar modo abierto
- Establecer modo WiFi como AP
- Establecer configuración AP
- Iniciar WiFi
- Loggear datos de AP

init_sta(ssid, password):

- Inicializar pila de red y eventos
- Crear interfaz WiFi en modo STA
- Inicializar WiFi
- Registrar manejadores de eventos WiFi y IP
- Configurar estructura de STA con SSID y password
- Si el password está vacío, usar modo abierto
- Sino, usar autenticación WPA2
- Establecer modo WiFi como STA
- Establecer configuración STA
- Iniciar WiFi
- Loggear intento de conexión

wifi_save_config(modos, ssid, password):

- Abrir NVS en modo escritura
- Guardar modos, ssid y password
- Confirmar cambios
- Cerrar NVS
- Retornar resultado

wifi_load_config(modos, ssid, password):

- Abrir NVS en modo lectura
- Leer modos, ssid y password

```
Mostrar configuración cargada
Cerrar NVS
Retornar resultado

wifi_clear_config:
  Abrir NVS en modo escritura
  Borrar todos los datos del espacio de nombres
  Confirmar cambios
  Cerrar NVS
  Retornar resultado

wifi_reset_retry_count:
  Reiniciar contador de reintentos

wifi_init_from_flash:
  Cargar configuración desde NVS (modo, ssid, password)
  Si hay error:
    Retornar error
  Si el modo es AP:
    Llamar a init_ap con ssid y password cargados
  Si el modo es STA:
    Llamar a init_sta con ssid y password cargados
  Si el modo es inválido:
    Retornar error
  Retornar ESP_OK

init_wifi:
  Llamar a wifi_init_from_flash
  Si hubo error cargando configuración desde flash:
    Inicializar WiFi en modo AP con SSID "CALIOPE 2.0"
    Guardar esta configuración en flash
  Sino:
    Mostrar mensaje de inicialización correcta desde flash
  Iniciar servidor web (módulo http)
```

HTTP

Este componente gestiona un servidor web HTTP que proporciona una interfaz web completa para controlar el reproductor y configurar tanto la conectividad WiFi como la configuración MQTT. El servidor sirve archivos estáticos (HTML, CSS, JavaScript) embebidos en el firmware y maneja múltiples endpoints API para recibir comandos y configuraciones.

La interfaz web incluye controles multimedia visuales (play, pause, stop, next, prev, volumen), formularios para configurar WiFi en modo estación (STA) o punto de acceso (AP), y formularios para configurar parámetros MQTT. Todas las configuraciones se guardan persistentemente en la memoria flash (NVS) y provocan un reinicio automático del dispositivo para aplicar los cambios.

Estructura del código:

Definir variables globales para:
servidor HTTP (httpd_handle_t)
referencias a archivos embebidos (HTML, CSS, JS)
buffer de datos POST y tamaño máximo

url_decode(destino, origen):
Decodificar strings URL-encoded recibidos desde formularios web
Convertir caracteres especiales (%20 → espacio, + → espacio)
Manejar códigos hexadecimales de caracteres especiales

restart_task():
Mostrar mensaje de reinicio en 3 segundos
Esperar delay de 3 segundos
Llamar esp_restart() para reiniciar el dispositivo

http_get_handler_html(request):
Servir archivo index.html embebido

http_get_handler_js(request):
Servir archivo script.js embebido

http_get_handler_css(request):
Servir archivo style.css embebido

http_comando_handler(request):
Recibir comandos POST desde controles multimedia web
Extraer parámetro "comando" del body POST
Según comando recibido:
"play": Ejecutar player_send_cmd(CMD_PLAY)
"pause": Ejecutar player_send_cmd(CMD_PAUSE)
"stop": Ejecutar player_send_cmd(CMD_STOP)
"next": Ejecutar player_send_cmd(CMD_NEXT)
"prev": Ejecutar player_send_cmd(CMD_PREV)
"volup": Incrementar volumen en 5 usando player_set_volume()
"voldown": Decrementar volumen en 5 usando player_set_volume()
Enviar respuesta con confirmación de ejecución

http_sta_handler(request):
Recibir configuración WiFi para modo estación (STA)
Extraer parámetros "ssid" y "password" del POST
Decodificar parámetros URL-encoded
Validar que SSID y password no estén vacíos
Guardar configuración WiFi en flash
Crear tarea restart_task para reiniciar en 3 segundos

```

http_ap_handler(request):
    Recibir configuración WiFi para modo punto de acceso (AP)
    Extraer parámetros "ssid" y "password" del POST
    Decodificar parámetros URL-encoded
    Validar que SSID y password no estén vacíos
    Guardar configuración WiFi en flash
    Crear tarea restart_task para reiniciar en 3 segundos

http_mqtt_handler(request):
    Recibir configuración MQTT
    Extraer parámetros: "broker", "puerto", "topic_evento", "topic_buffer"
    Decodificar todos los parámetros URL-encoded
    Guardar configuración MQTT usando mqtt_save_config()
    Crear tarea restart_task para aplicar cambios

http_get_wifi_config_handler(request): API para obtener configuración actual
    Cargar configuración WiFi actual desde flash
    Cargar configuración MQTT actual desde flash
    Crear respuesta JSON con estructura:
        wifi: {ssid, password, mode, configured}
        mqtt: {broker, puerto, topic_evento, topic_buffer, configured}
    Enviar JSON como respuesta

start_webserver():
    Verificar si servidor ya está iniciado
    Configurar httpd_config_t con puerto 80
    Iniciar servidor HTTP usando httpd_start()
    Si éxito:
        Registrar GET "/" → http_get_handler_html
        Registrar GET "/script.js" → http_get_handler_js
        Registrar GET "/style.css" → http_get_handler_css
        Registrar POST "/comando" → http_comando_handler
        Registrar POST "/sta" → http_sta_handler
        Registrar POST "/ap" → http_ap_handler
        Registrar POST "/mqtt" → http_mqtt_handler
        Registrar GET "/api/wifi-config" → http_get_wifi_config_handler
    Si error: Loggear error

```

NTP

Este gestiona la sincronización de tiempo del dispositivo con varios servidores NTP para obtener timestamps precisos. El módulo utiliza el protocolo SNTP para conectarse a múltiples servidores de tiempo públicos y mantener la hora del sistema sincronizada.

El componente proporciona funciones para inicializar, iniciar y detener la sincronización NTP, así como para obtener timestamps en formato ISO (YYYY-MM-DDTHH:MM:SSZ). Mantiene un estado interno de sincronización que le permite a los otros componentes (como MQTT y Logger) verificar si el tiempo está sincronizado.

La sincronización se realiza automáticamente cada hora una vez iniciada.

La estructura quedo de la siguiente manera:

`ntp_sync_callback(timeval)`: Callback ejecutado cuando se completa sincronización NTP

`ntp_init()`: Inicializar sincronización NTP

`ntp_start()`: Iniciar sincronización NTP activa

`ntp_stop()`: Detener sincronización NTP

`ntp_synced()`: Verificar estado de sincronización NTP

`ntp_time(buffer, tamaño_buffer)`: Obtener timestamp actual en formato ISO

`ntp_print_status()`: Función de debug para mostrar estado completo

Bibliografía

- *Technical Documents. Espressif*: <https://www.espressif.com/en/support/documents/technical-documents>
- *FreeRTOS Documentation*. <https://www.freertos.org/>